

TP No. 8

Sorting and search algorithms

Exercise 1:

The sequential search algorithm (or linear) consists of examining the table element by element and seeing whether or not info belongs to table T. If the result is positive; this algorithm returns the index of the first occurrence of the info; otherwise it returns -1.

Implement the int SequentialSearch (int T [], int n, int info) function in C

Exercise 2:

Consider the following program:

```
# include <stdio.h>
# define n 5
float a [n]
float info;
unsigned P (unsigned x) {
    return a[x] == info ;
}
void main(void){
    unsigned x ;
    unsigned y ;
    unsigned p, q ;
    unsigned i ;
    for (i=0; i<n; i++){
        printf("give element num %u \n", i);
        scanf("%f", &a[x]) ;
    }
    printf("give real \n");
    scanf("%f", &info) ;

    p=0 ;
    q=n ;
    x=p ;
    y=q ;
    while(x !=y) {
        if(P(x))
            y=x ;
        else
            x++ ;
    }
    printf("%u\n", x !=q) ;
}
```

Questions :

- Explain the general organization of the program cited above.
- Identify the problem supposed to be solved by the previous program and compare the proposed solution with algorithms solving the same type of problem.

Exercise 3:

The selection sort technique is the simplest technique, it consists of:

- find the index of the smallest element of the array $T[0..n-1]$ and swap the corresponding element with the element of index 0
- find the index of the smallest element of the array $T[1..n-1]$ and swap the corresponding element with the element of index 1
- ...
- find the index of the smallest element of the array $T[n-2..n-1]$ and swap the corresponding element with the element of index (n-2).

1) Write the function void TriSelection(int T[], int n) which allows the sorting of the first n elements of the integer array T.

2) Write a main program (the main function) that reads the dimension n of an array T of type int (maximum dimension: 50 components), fills this array with values entered on the keyboard and displays the array. Then, sort this array using the TriSelection function and redisplay this array.

Exercise 4:

This method consists of taking the elements of the array one by one and inserting each one in its correct place so that the processed elements form a sorted sublist.

To do this, proceed as follows:

- compare and swap if necessary $T[0]$ and $T[1]$ so as to place the smallest in the index box 0
- compare and swap if necessary the element $T[2]$ with those which precede it in order ($T[1]$ then $T[0]$) in order to form a sorted sub-list $T[0..2]$
- ...
- compare and swap if necessary the element $T[n-1]$ with those which precede it in order ($T[n-2]$, $T[n-3]$, ...) in order to obtain a sorted table.

1) Write the function void TriInsertion(int T[], int n) which allows the sorting of the first n elements of the integer array T.

2) Write a main program (the main function) that reads the dimension n of an array T of type int (maximum dimension: 50 components), fills this array with values entered on the keyboard and displays the array. Then, sort this array using the InsertSort function and redisplay this array.

Exercise 5:

Compare the two algorithms selection sort and insertion sort.

Exercise 6:

Create an algorithm in C that allows you to calculate the number of subsequences and the longest strictly increasing subsequence in an array of n integers.

Exercise 7:

We give an array called redundant containing redundant integers and in any order.

Informally present and implement in C an algorithm for counting the frequency of each element appearing in the redundant array in a table.

Exercise 8:

We have a table of whole numbers, classified by increasing value, each whole number can be repeated several times: 3 5 5 5 7 9 9 9 9 10 10 12 12 12 13

A board is the sequence of all consecutive integers with the same value: in the example below, the board with value 5 contains 3 numbers, the board with value 10 contains 2, etc. The length of a board is called the value common to all its terms. Given an array of n terms, find the length and value of its longest board.

Exercise 9:

We are given a table containing pairs (x, y) where x is a real information type and y its frequency (unsigned integer information type). Such a table is sorted in ascending order on the field x. In addition, we define the following operations:

1. search: it allows you to see whether a piece of information (info) of real type belongs or not to the table. If the result is positive, search also returns the position of this element. Program this operation in C.
2. increment: this allows you to increment the frequency relative to x already located. Program this operation in C.
3. add: it adds x with zero frequency to the table. It leaves the table sorted in ascending order on the x field.

Give the signature of this operation and informally the appropriate algorithm for programming this operation.

Exercise 10:

Let T be an array of n integers. The bubble sort method requires two steps:

- Iterate through the array elements from 0 to (n-2); if element i is greater than element (i+1), then they are swapped.

- The program stops when no permutation is possible after a complete scan of the table.

1) Write the function void TriBulles (int T [], int n) which allows the sorting of the first n elements of the integer array T.

2) Write a main program (the main function) that reads the dimension N of an array T of type int (maximum dimension: 50 components), fills this array with values entered on the keyboard and displays the array. Then, sort this array using the TriBulles function and redisplay this array.

Exercise 11:

The dichotomous search technique is only applicable if the table is already sorted (e.g., in ascending order). The goal of dichotomous search is to divide the search interval by 2 at each iteration. To do this, proceed as follows:

Let first and last be the left and right ends of the interval in which we are looking for the value x, we calculate M, the index of the median element:

$$M = (\text{first} + \text{last}) \text{ div } 2$$

There are 3 possible cases:

- $x = T[M]$: the element with value x is found, the search is finished
- $x < T[M]$: the element x, if it exists, is in the interval [prime..M-1]
- $x > T[M]$: the element x, if it exists, is in the interval [M+1..last]

Binary search involves iterating this process until x is found or the search interval is empty.